ASP.NET Web Application

To open ASP.NET Web Application:

1. open visual studio.
2. Click on open new project.
3. Choose ASP.NET Web Application (.NET Framework).
4. Click next.
5. Choose empty and mark on MVC.
6. Click create.

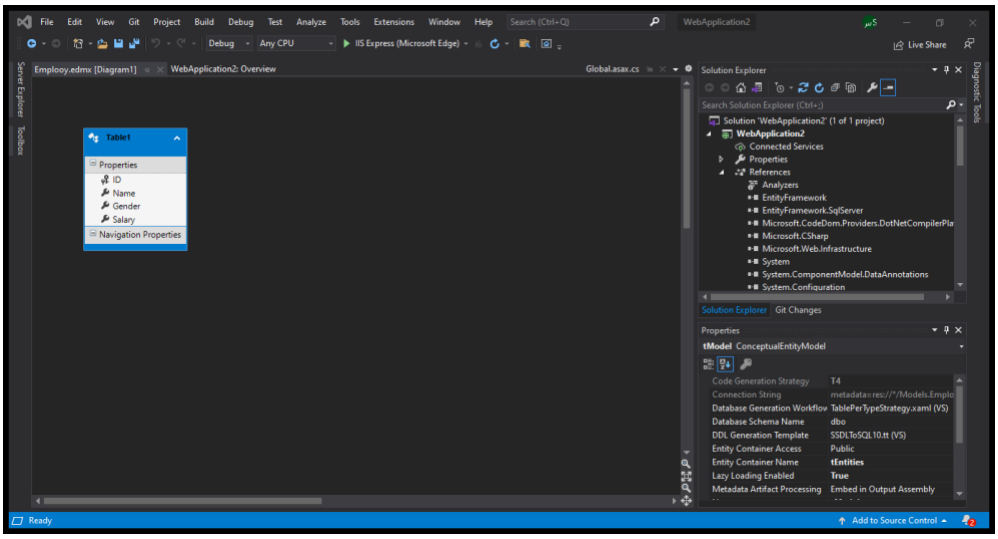
To create database:

1. open Microsoft SQL server management studio.
2. Create a database.
    - Right Click on database.
    - Choose new database.
    - Enter the database name.
    - Click ok.
3. Choose the name of database that you create.
4. Create table.
    - Right Click on the table.
    - Choose new.
    - From new Choose table.
    - Enter columns name & data type.
    - Stand on column that you want to make it primary key and Right Click on it then choose set primary key.
    - Stand on column that you want to increment it then go to its properties choose identify specification then make (Is Identity) yes then in Identity increment enter the number you want to increment by it.
    - Finally click save to save the table & enter the table name.

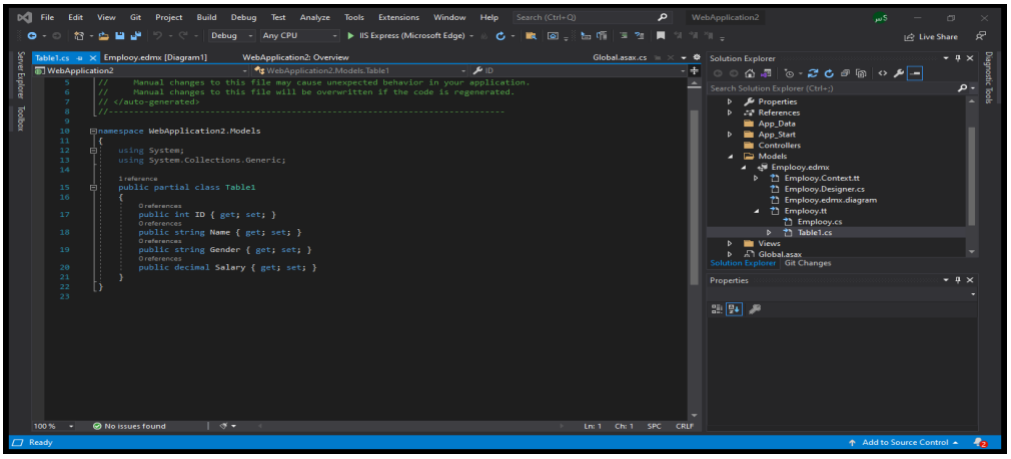
To connect ASP.NET Web Application with database:

1. After open visual studio.
2. Go to Solution Explorer.
3. Right click on Models folder.
4. Select Add then click new item.
5. In installed→ visual C# → Data choose ADO.NET Entity Data Model & give it name then click add.
6. Click next if the select is (EF Designer from data…).

7. Click on new connection.
8. Enter server name then select the database name then click Ok.
9. Click next.
10. Then click next again.
11. Then select tables you want then click finish.
12. When appear security page click ok.
13. When the following table appear.

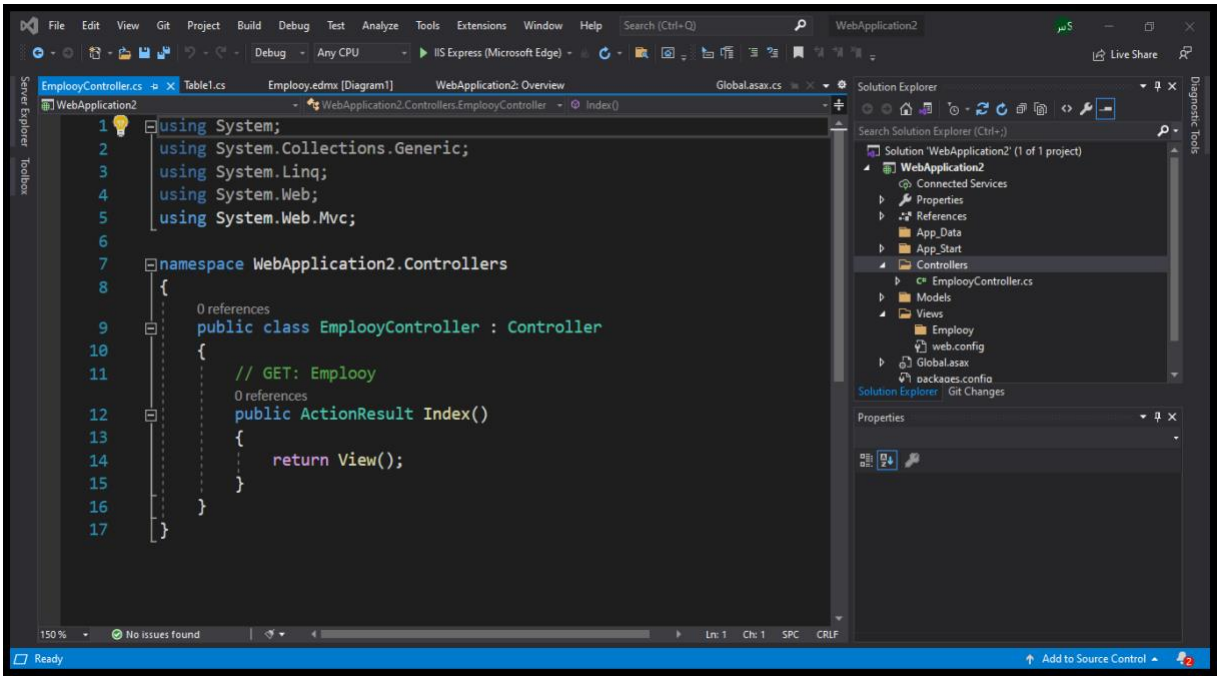


14. Then select Build then click build solution.
15. After make build go to Solution Explorer.
16. Choose Models → click on your table name in your database.
17. So this appear.



To create controller:

1. Go to Solution Explorer.
2. Right click on Controller folder.
3. Select Add then click Controller.
4. Select (MVC5 Controller_ Empty) then click add.
5. Enter controller name then click add.



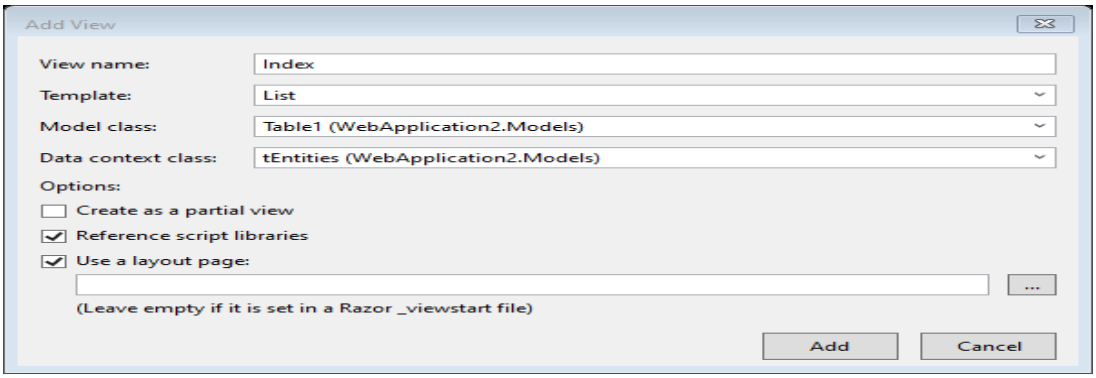
To connect controller with database:

1. Go to Solution Explorer.
2. Click on web.config folder.
3. Go to search in find & replace search about connection.
4. Click find next .
5. From the connection string take the name to make controller connected with database.

Each method return view in controller must make to it view.

To make view:

1. Right click in the name of method.
2. Click add view.
3. Click add.
4. Enter view name as the name of method & the template as you want then enter all data as following.



5. Click add.

Code of controller:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using WebApplication2.Models;

namespace WebApplication2.Controllers
{
    public class EmplooyController : Controller
    {
        tEntities db = new tEntities();
        // Index method
        public ActionResult Index()
        {
            return View(db.Table1.ToList ( ));
        }
        // Create method
        [HttpGet]
        public ActionResult Create()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Create(Table1 a)
        {
            //cheak if constrains verifier or not.
            if (ModelState.IsValid)
            {
                db.Table1.Add(a);
                db.SaveChanges();
                return RedirectToAction("Index");
            }
            return View(a);
        }
        //Edit method
        [HttpGet]
        public ActionResult Edit(int Id)
        {
            Table1 a = db.Table1.Find(Id);
            return View(a);
        }
        [HttpPost]
        public ActionResult Edit(Table1 a)
        {
            //cheak if constrains verifier or not.
            if (ModelState.IsValid)
            {
                db.Entry(a).State = System.Data.Entity.EntityState.Modified;
                db.SaveChanges();
                return RedirectToAction("Index");
            }
            return View();
        }
        //Delete method

        [HttpGet]
        public ActionResult Delete(int Id)
        {
            Table1 a = db.Table1.Find(Id);
            return View(a);
        }
```

```
        // where the method of the Delete not same name with ConfirmDeleteso you shoud do the following
in [HttpPost]
        [HttpPost,ActionName("Delete")]
        public ActionResult ConfirmDelete(int Id)
        {
            //cheak if constrains verifier or not.
            if (ModelState.IsValid)
            {
                Table1 a = db.Table1.Find(Id);
                db.Table1.Remove(a);
                db.SaveChanges();
                return RedirectToAction("Index");
            }
            return View();
        }
        // anoter method to delete
        //public ActionResult Delete(int Id)
        //{
        //    Table1 a = db.Table1.Find(Id);
        //    db.Table1.Remove(a);
        //    db.SaveChanges();
        //    return RedirectToAction("Index");
        //}

    }
}
```

IF we create a database, its name called sign has table called Admin

Code of controller:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using WebApplication4.Models;

namespace WebApplication4.Controllers
{
    public class HomeController : Controller
    {
        // GET: Home
        signEntities db = new signEntities();
        public ActionResult Index()
        {
            return View(db.Admins.ToList());
        }
         //SignUp method--> template is create
        [HttpGet]
        public ActionResult SignUp()
        {
            return View();
        }
        [HttpPost]
        public ActionResult SignUp(Admin a)
        {
            if (ModelState.IsValid)
            {
                db.Admins.Add(a);
                db.SaveChanges();
            }
            return View();
```

```
        }
        //Login method--> template is create
        [HttpGet]
        public ActionResult Login()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Login(Admin user)
        {
            // (a=>)is an object but in it name that found in your data
            var result= db.Admins.Where(a => a.Name == user.Name &&
a.Passward==user.Passward).FirstOrDefault();
            if (result != null)
            {
                Session["x"] = "Hello " + user.Name;
                return RedirectToAction("Index");
            }
            else
            {
                ModelState.AddModelError("", "In Valid");
            }
            return View();
        }
    }
}
```

Code of Admin.cs:

```
namespace WebApplication4.Models
{
    using System;
    using System.Collections.Generic;
    using System.ComponentModel.DataAnnotations;
    public partial class Admin
    {
        public int ID { get; set; }
        [Required(ErrorMessage = "Enter your Name")]
        public string Name { get; set; }
        [Required(ErrorMessage = "Enter your E_mail")]
        [DataType(DataType.EmailAddress)]
        public string E_mail { get; set; }
        [Required(ErrorMessage = "Enter your Passward")]
        [DataType(DataType.Password)]
        public string Passward { get; set; }

    }
}
```